

# Amptek Mini-X Controller — Communication Protocol

**Reverse-engineered from the Amptek Mini-X DLL source (MiniXDlg.cpp, MiniXDlg.h) and checked against hardware.**

|                  |                                                                                                                                                                                  |
|------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Document date    | 13 August 2026; revised 17 September 2026                                                                                                                                        |
| Reference source | Amptek Mini-X DLL source, MiniXDlg.cpp / MiniXDlg.h, board revision C0. Kept locally in reference/; not in the repository, because it is proprietary.                            |
| Validated on     | Mini-X Controller, serial 01300036, 50 kV board, 10 W rating (from the unit’s hardware documentation, not from software); macOS, pyftdi                                          |
| Status           | DAC, ADC, GPIO and HV enable confirmed on hardware (§10). The DS1722 read and configuration write are confirmed (§8.4). Nothing has been exercised near the power limit (§10.5). |

**About this revision.** The first version was reconstructed from an excerpt of MiniXDlg.cpp. This revision checks it against the full source and against hardware runs of this project’s code; the run log is in [hardware-notes.md](#). §13 lists what changed. Where it matters, the text says whether a statement comes from the **source**, from **hardware** observation, or is this project’s **recommendation**. A recommendation is not vendor behaviour.

## 1. Overview

The Mini-X controller is a USB device built around an **FTDI FT2232C/D** in **MPSSE** (Multi-Protocol Synchronous Serial Engine) mode. There is no packet-oriented command language: the host synthesizes bit-level SPI-like waveforms by writing MPSSE opcode streams, and the FTDI chip drives the peripherals directly.

Four peripherals share the FT2232’s channel-A pins:

| Peripheral  | Part                                       | Function                      | Chip select       | Polarity           |
|-------------|--------------------------------------------|-------------------------------|-------------------|--------------------|
| Dual DAC    | AD5623R                                    | HV and current setpoints      | DACS (ADBUS4)     | Active <b>low</b>  |
| ADC         | 12-bit, 2-channel; part unconfirmed (§6.3) | HV and current monitors       | ADCS (ADBUS3)     | Active <b>low</b>  |
| Temp sensor | DS1722                                     | Board temperature             | TSCS (ACBUS3)     | Active <b>high</b> |
| GPIO        | —                                          | HV enables, interlock, status | direct pin access | see §7             |

All four share a single clock and a single bidirectional data line. The host manages the chip selects; there is no hardware arbitration. **All transactions must be serialized** (§11).

Source comments in the ADC and DS1722 routines say the clock is inverted on its way to those parts (“take ADC clock enable low – this makes clock high at the A/D”). This is why each peripheral uses a different clock idle level, and why the levels must not be normalized (§6.2, §6.3, §8.3).

## 2. USB identification

| Field         | Value                                     |
|---------------|-------------------------------------------|
| Vendor ID     | 0x0403 (FTDI)                             |
| Product ID    | 0xD058, a <b>custom Amptek PID</b>        |
| Description   | MiniX Controller                          |
| Serial number | 8 ASCII digits, e.g. 01300036             |
| Interface     | Channel A (/1 in pyftdi URL syntax)       |
| bcdDevice     | 0x0500 (FT2232C/D); confirmed on hardware |

Generic FTDI tools that filter on known PIDs will **not** enumerate this device until the PID is registered:

```
Ftdi.add_custom_product(0x0403, 0xD058, 'minix')
```

**Source.** The reference never uses the VID/PID. It lists FTD2XX devices by description, proceeds only if more than one is listed (channels A and B each appear), and opens the first by description. FTD2XX appends a " A" / " B" channel suffix, which the reference strips for display (ChCount -= 2); pyftdi reports the base string. The serial number comes from FT\_ListDevices by index 0: the first 8 characters, converted with atol.

### 2.1 Serial number selects the calibration table

**Source** (GetDeviceSerialNumber, OnInitDialog):

```
at dialog init          -> 40 kV table (ReadMiniXSetup)
serial > 9999           -> isNSI = true
    and serial >= 1118880 -> is50kv = true, 50 kV table (ReadMiniXSetup50kv)
    otherwise           -> 40 kV table
serial <= 9999          -> isNSI = false ("Comet" in the source);
                        keeps the 40 kV table from init
```

Serial 01300036 → 1300036 → **NSI, 50 kV**.

isNSI selects the setpoint and shutdown timing (§9.2). is50kv selects HighVoltageMax, HighVoltageConversionFactor, and which iso-curve bitmap is shown.

**The serial number does not encode the power rating.** Both setup routines contain WattageMax = 4.00 as a literal. No serial-number threshold, strapping pin or register reports the rating. Voltage range and power rating are **independent** settings, and only the voltage range can be discovered at runtime.

**The rating must therefore come from outside the protocol:** the hardware label, the shipping documentation, or the vendor. Do not infer it. See §6.1.1.

*Unexplored:* MiniXDlg.h declares EE\_Read / EE\_Program wrappers for the FT2232 EEPROM, which the reference never uses for configuration. Whether the vendor stores a rating in the EEPROM user area is unknown. A read-only dump would settle it; until then, treat the EEPROM as a curiosity, not a source of truth.

## 3. Pin map (revision C0)

**ADBUS (low byte)**

| Bit | Mask | Name  | Dir | Function                    |
|-----|------|-------|-----|-----------------------------|
| 7   | 0x80 | MONX  | In  | MON MINIX RDY status (§7.2) |
| 6   | 0x40 | HVEN2 | Out | CTRL_HV_EN_B                |
| 5   | 0x20 | HVEN  | Out | CTRL_HV_EN_A                |

| Bit | Mask | Name    | Dir | Function                     |
|-----|------|---------|-----|------------------------------|
| 4   | 0x10 | DACS    | Out | DAC chip select (active low) |
| 3   | 0x08 | ADCS    | Out | ADC chip select (active low) |
| 2   | 0x04 | DataIN  | In  | MISO                         |
| 1   | 0x02 | DataOUT | Out | MOSI (DATASTATE)             |
| 0   | 0x01 | CLK     | Out | SCLK (CLKSTATE)              |

Direction byte: **OUTPUTMODE = INPUTMODE = 0x7B** (0111 1011).

The two constants are identical in revision C0, so the direction byte never changes. The **INPUTMODE** writes in the read paths only rewrite the state byte. Earlier board revisions (A0/A1/B0) used different values and did turn the data line around; the source covers them, this document does not.

**Hardware.** Bits 0–1 read back whatever clock and data levels the previous transaction left: ...1111 after init, ...1110 after an ADC read, ...1100 after a DS1722 read. They carry no status.

#### ACBUS (high byte)

| Bit | Mask | Name   | Dir | Function                                      |
|-----|------|--------|-----|-----------------------------------------------|
| 3   | 0x08 | TSCS   | Out | Temp sensor chip enable (active <b>high</b> ) |
| 2   | 0x04 | ?      | In  | <b>Unidentified</b> ; always read set         |
| 1   | 0x02 | —      | —   | not used                                      |
| 0   | 0x01 | !RESET | In  | Interlock: 1 = closed, 0 = open (§7.3)        |

Direction byte: **OUTPUTMODE\_H = 0x08**. Only ACBUS3 is an output.

**Hardware.** The FT2232C/D has only ACBUS0–3 on channel A. Bits 4–7 of a 0x83 read are not pins, and have been observed set at random (0x45, 0x85). **Mask the byte with 0x0F.**

## 4. MPSSE opcodes used

| Opcode | Operands            | Meaning                           | Used by                  |
|--------|---------------------|-----------------------------------|--------------------------|
| 0x80   | state, dir          | Set ADBUS data + direction        | all                      |
| 0x82   | state, dir          | Set ACBUS data + direction        | init, DS1722             |
| 0x81   | —                   | Read ADBUS (returns 1 byte)       | GPIO                     |
| 0x83   | —                   | Read ACBUS (returns 1 byte)       | GPIO                     |
| 0x86   | 0x03 0x00           | Set clock divisor → ~1.5 MHz      | ADC, DS1722, startup     |
| 0x10   | lenL, lenH, data... | Clock <b>bytes</b> out, MSB first | DAC, DS1722 config write |
| 0x12   | len, data           | Clock <b>bits</b> out, MSB first  | DS1722 address (§8.3)    |
| 0x13   | len, data           | Clock <b>bits</b> out, MSB first  | ADC control nibble       |
| 0x20   | lenL, lenH          | Clock <b>bytes</b> in, MSB first  | ADC, DS1722              |

0x10/0x11, 0x12/0x13 and 0x20/0x24 differ only in clock edge. The edge each path needs was found on hardware and differs between peripherals (§10.1, §8.4). Length fields are *n-1*: 0x02 0x00 = 3 bytes, 0x01 0x00 = 2 bytes; for bit mode, 0x03 = 4 bits and 0x07 = 8 bits.

On the FT2232C/D, SCK = 12 MHz / ((1 + divisor) × 2), so divisor 3 gives 1.5 MHz. Do not send FT2232H-only opcodes (0x8A, 0x8D, 0x97 and others). The FT2232C/D rejects them, and each rejection leaves two bytes in the RX buffer (§5.1).

**Naming caveat (source).** The source defines CLK\_FN\_NEG 0x10 and CLK\_FN\_POS 0x11, then selects CLK\_FN = CLK\_FN\_NEG. The names and inline comments contradict each other and FTDI AN\_108. The numeric opcodes are authoritative; ignore the edge annotations in the source.

**Invalid alternatives (source).** ReadTemperature() has commented-out alternatives 0x22, 0x24, 0x26 next to 0x20. 0x22 and 0x26 are *bit-mode* reads that take a **single** length byte. Written with two length bytes, as in the source, they would leave a stray 0x00 to be parsed as a command, which MPSSE rejects. Only 0x20 and 0x24 are valid byte-mode reads there.

## 5. Initialization

**Source** (OnButtonMpsseOn):

```
1. FT_SetLatencyTimer(4)           # 4 ms
2. FT_SetTimeouts(40, 40)         # ms read/write
3. FT_SetBitMode(0x00, 0x02)      # enable MPSSE
4. 0x82, 0x00, 0x08               # ACBUS: TSCS deasserted (low)
   0x80, 0xFB & ~0x20 & ~0x40 (= 0x9B), 0x7B # ADBUS: HV enables CLEARED
```

Step 4 is one write. It is the safety-critical step: the ADBUS state starts from 0xFB (all outputs high) with **both HV enable bits explicitly cleared**, so bringing up the link cannot energize the tube.

- **Clock divisor.** It is not part of this routine; SetTempSensor() and SetClkDivisor() send it next (§5.2).
- **Purging.** The reference defines a Purge wrapper but **never calls it**.
- **This project** purges before every transaction whose reply it parses (§11), sends the divisor with the initial pin setup, and runs the sync check (§5.1) before trusting any read.

To tear down: FT\_SetBitMode(0x00, 0x00).

### 5.1 Health check

Not in the reference. Sending an invalid opcode makes MPSSE reply 0xFA followed by the offending byte. Confirmed on hardware:

send 0xAB → receive 0xFA 0xAB

This verifies synchronization before any read is trusted.

### 5.2 Full startup sequence

**Source** (OnBnClickedStartMiniXController). The block runs **twice**:

```
select and open device (§2), read serial, select table (§2.1)
if open:
    200 ms
    MPSSE init (§5)
    SetTempSensor (§8.3)
    SetClkDivisor (0x86 0x03 0x00)
    HV DAC = 0
    NSI only: 200 ms
```

```

current DAC = 0
show "Connecting to MiniX. Please wait..."; 2 s
repeat the block above, then start the 1 s monitor timer (§6.4)

```

The source gives no reason for the repetition. `SetTempSensor()` sends its own clock divisor, so the following `SetClkDivisor()` is redundant but harmless. Writing zero to the DACs is safe at this point, because the enables were cleared in step 4.

## 6. Analog channels

### 6.1 Scaling constants

Three parameters vary between units. The serial number selects two of them; the third, the power rating, is not discoverable at runtime.

#### Voltage range (from `is50kv`)

| Constant                                 | 40 kV board | 50 kV board |
|------------------------------------------|-------------|-------------|
| <code>HighVoltageConversionFactor</code> | 10.0 kV/V   | 12.5 kV/V   |
| <code>HighVoltageMax</code>              | 40.0        | 50.0        |
| <code>HighVoltageMin</code>              | 9.999999    | 9.999999    |

#### The same on every unit

| Constant                             | Value                                                                                            |
|--------------------------------------|--------------------------------------------------------------------------------------------------|
| <code>VRef</code>                    | 4.096 V                                                                                          |
| <code>DAC_ADC_Scale</code>           | 4096 (12-bit)                                                                                    |
| <code>CurrentConversionFactor</code> | 50.0 $\mu$ A/V                                                                                   |
| <code>CurrentMin</code>              | 4.999999 (0.999999 in a <code>_RELEASE_1uA</code> build, 50 kV table only)                       |
| <code>CurrentMax</code>              | 200.0                                                                                            |
| <code>SafetyMargin</code>            | 0.050 W                                                                                          |
| Default setpoints                    | 15 kV, 15 $\mu$ A (the commented-out INI templates say 10 kV / 5 $\mu$ A; the code uses 15 / 15) |

**Nothing in the DAC or ADC path varies with the power rating.** Full scale is 4.0 V (4000 counts) on both channels regardless: 50 kV and 200  $\mu$ A, whose product is exactly 10 W. Resolution is 0.0125 kV and 0.05  $\mu$ A per count. The analog path is sized for the higher rating on every unit; only the software clamp differs.

The minima are stored as 9.999999 and 4.999999, not 10.0 and 5.0. Values are clamped against the minimum and then formatted "`%.0f`", so the epsilon lets an entry of exactly 5.0 pass the `<` comparison while still displaying as "5".

**6.1.1 Power rating** The vendor ships both 4 W and 10 W variants. **The rating is a compile-time literal in the reference and cannot be discovered at runtime (§2.1).**

| Constant                      | 4 W variant | 10 W variant                   |
|-------------------------------|-------------|--------------------------------|
| <code>WattageMax</code>       | 4.00 W      | 10.00 W                        |
| <code>SafetyMargin</code>     | 0.050 W     | 0.050 W as shipped (see below) |
| <code>SafeWattageMW</code>    | 3950 mW     | 9950 mW                        |
| <code>mwCaution (§6.4)</code> | 3900 mW     | 9900 mW                        |

| Constant                 | 4 W variant | 10 W variant |
|--------------------------|-------------|--------------|
| mwDanger (§6.4)          | 4000 mW     | 10000 mW     |
| WattInRange limit (§6.4) | 4050 mW     | 10050 mW     |

The source contains only the 4 W values. The 10 W column substitutes  $\text{WattageMax} = 10.00$  into the same expressions; **it is derived, not transcribed.**

**The safety margin is absolute, not proportional.** 50 mW is 1.25 % of a 4 W rating but 0.5 % of a 10 W rating, while the measurement uncertainty stays the same (+4 % on the current channel at low setpoints, §10.3). Scaling the margin with the rating (125 mW at 10 W keeps 1.25 %) is an option. Whichever you choose, document it, because the two variants will then disagree on where the caution band begins.

**The two possible mistakes are not equivalent:**

- Configuring **10 W on a 4 W unit** lets the operator command  $2.5\times$  the tube's rating. The software clamp is the only thing preventing it: the analog path will comply, and the range check (§6.4) compares monitors with setpoints, not setpoints with the rating.
- Configuring **4 W on a 10 W unit** costs flux and nothing else.

**When the rating is not confirmed, assume 4.00 W.**

**Recommendation.** Carry no default. Require the rating at construction, accept only 4.0 or 10.0, and refuse to open the device otherwise. Store the rating with the serial number in the configuration file. **Display it persistently** next to the power indicator, log it at startup, and include it in every run record.

The source's own history supports a configuration file. A commented-out `ReadMiniXSetup()` once read `WattageMax`, `Safety Margin`, `High Voltage kV Max` and the conversion factors from `MiniXCtrl.ini`; the two hardcoded setup functions replaced it.

## 6.2 DAC write (AD5623R)

Command bytes: 2 unused bits, 3-bit command, 3-bit address (XXCCCAA).

| Channel     | Byte | Function               |
|-------------|------|------------------------|
| A (HV)      | 0x18 | Write and update DAC A |
| B (current) | 0x19 | Write and update DAC B |

Both are write-**and-update** (confirmed on hardware): values take effect without a separate update command.

```
counts = (int)(volts / VRef * 4096)           # truncates toward zero
bhi     = (counts & 0x0FF0) >> 4
blo     = (counts & 0x000F) << 4             # 12 bits left-justified
```

Byte stream:

```
0x80, state & ~DACS | CLKSTATE, 0x7B      # assert DACS, CLK high
0x10, 0x02, 0x00, <0x18|0x19>, bhi, blo  # clock out 3 bytes
0x80, state | DACS, 0x7B                  # deassert DACS
```

The state byte sets `CLKSTATE` **high** as `DACS` is asserted, the reverse of the ADC path. The source has the opposite operation commented out directly above, which suggests this was found by experiment. Confirmed on hardware.

**How a setpoint reaches the DAC (source).** Each per-channel handler runs in this order. The handlers are `OnBnClickedSethightvoltagecontrolbutton` and `OnBnClickedSetcurrentcontrolbutton`.

1. Read the input field and clamp it to `[min, max]`.
2. Add 0.00001 and set `PrevVoltage / PrevCurrent` to the integer part.

3. Show the value in the field as "%.0f".
4. Divide by the conversion factor, format as "%.4f", and pass the result to the DAC routine.

The DAC gets the **unrounded** value (to 0.1 count); only the display is rounded. But see §9.1 for when the field text itself is rounded before it is read.

**The DAC routine refuses out-of-range values while HV is on (source).** With HV0n set, a value outside [min, max] / factor is not written. The routine sets a status string and returns, and the DAC keeps its old value. For the HV channel the window widens by  $\pm 2.5\%$  during the non-NSI correction step (§9.2). HV-off clears HV0n before writing zeros, so those writes always go through.

**This project** adds  $1e-9$  counts before truncating. Without it, binary float error drops a count at round setpoints (10.1 kV becomes 807 counts instead of 808).

### 6.3 ADC read

**Part identity.** The source and the first version of this document call the ADC “MAX186-family”. A MAX186 takes an 8-bit control byte and outputs zeros after the data word. The part on this board takes a 4-bit control word and **repeats the data LSB-first after the LSB** (hardware, see Unpacking). That behaviour matches an MCP3202/LTC1298-type converter. The part number is unconfirmed, and it does not change the implementation.

The control nibble is clocked out 4 bits MSB-first as Start, SGL/DIF, ODD/SIGN, MSBF:

| Constant | Byte | Nibble clocked | Channel | Monitors     |
|----------|------|----------------|---------|--------------|
| AD0      | 0xD0 | 1101           | 0       | High voltage |
| AD1      | 0xF0 | 1111           | 1       | Current      |

Only the upper 4 bits are clocked (0x13 with length 0x03); the low nibble is padding. Start, SGL/DIF and MSBF are the same in both words, so **ODD/SIGN alone selects the channel**. That is why a mis-clocked nibble produces the *other channel's* value rather than noise (§10.1).

Byte stream, identical for both channels apart from the nibble:

```
0x86, 0x03, 0x00          # clock divisor, re-sent on every read
0x80, state & ~ADCS & ~CLKSTATE, 0x7B # assert ADCS, CLK LOW
0x13, 0x03, <AD0|AD1>     # 4 bits out, MSB first
0x80, state, 0x7B         # "INPUTMODE": no-op on rev C0
0x20, 0x01, 0x00         # clock IN 2 bytes
0x80, state | ADCS, 0x7B  # deassert ADCS
```

Then read exactly 2 bytes.

- **CLKSTATE is cleared as ADCS is asserted**, the reverse of the DAC path. The source comment reads “take ADC clock enable low – this makes clock high at the A/D”. Do not normalize the two paths to a common idle level.
- **The INPUTMODE write is vestigial on rev C0**, because INPUTMODE == OUTPUTMODE. On A0/A1/B0 it turned the data line around. It costs one command; keep it.
- **No delay is needed.** The whole transaction is a single write; MPSSE runs it back to back and the two reply bytes wait in the RX buffer.
- **The reference re-applies SetTimeouts(40, 40) before each read**, redundantly, and does not purge.

**Unpacking** The reply is 1 null bit, then 12 data bits, then 3 trailing bits. **Hardware:** the trailing bits are not padding. They always equal data bits B1, B2, B3, because the converter continues LSB-first after the LSB. This held for every reading so far: the two §10.1 readings, 10 from a first read-only run and 400 from a 200-poll run, with no mismatches.

```
raw = ((rx[0] << 8) | rx[1]) >> 3          # 13 bits survive the shift
if raw & 0x1000:
```

```

    raise FramingError                                # null bit must be 0
counts = raw & 0x0FFF
b1_b3 = (counts >> 1 & 1) << 2 | (counts >> 2 & 1) << 1 | (counts >> 3 & 1)
if rx[1] & 0x07 != b1_b3:
    raise FramingError                                # trailing bits must repeat B1-B3
volts = counts / 4096 * 4.096

```

The reference computes the same 13-bit value but **checks neither condition**. With the null bit set it would report more than 4.096 V instead of flagging an error. Both checks are this project’s hardening. The trailing-bit check catches shifted frames that the null bit misses. **Neither check can detect a dead data line**, because an all-zero reply passes both.

Engineering units follow from §6.1:  $kV = volts \times HighVoltageConversionFactor$ ,  $\mu A = volts \times CurrentConversionFactor$ .

## 6.4 Monitor loop

**Source** (OnTimer, event 1). The monitor timer period is **1000 ms** (m\_nMonitorE lapse; a 2000 ms value is commented out). Each cycle runs these steps in order:

1. **HV channel.** Read the ADC, which yields a "%.4fv" string; convert it with `atof` and multiply by `HighVoltageConversionFactor`. This unrounded value is `HVCheck`. Add  $1e-9$ , format "%.0fkV", and `atoi` it: that integer is `iVoltsIn`. Feed the  $+1e-9$  value to the running average, displayed as "%.1fkV".
2. **Current channel.** The same steps give `ICheck`, `iMicroAmpsIn` and a "%.1fuA" average.
3. **Power.** `maWattage = iVoltsIn * iMicroAmpsIn`, from the rounded integers. This drives the colour band.
4. **Power limit.** `WattInRange = maWattage < (WattageMax + SafetyMargin) * 1000`.
5. **Range tests** on `HVCheck` / `ICheck`, as described under Range checking.
6. **Indicator.** An “out of range” indicator shows unless the power and both channels are in range.
7. **X-ray icon.** It blinks while the software flag `HVOn` is set.
8. **GPIO read and interlock handling** (§7).
9. **Temperature read** (§8).
10. **Averaging flag.** `isFirstMonitorAve = false`.

So the reference uses **three presentations** of each channel: the rounded integer (power and banding only), the unrounded value (range tests), and the 1-decimal average (display). A port that carries doubles throughout should compute the displayed power from doubles and say so; its figures will differ from the reference’s by up to a few tens of mW.

**Averaging.** `AveMonitors()` keeps a window of **NumAVE = 7** samples. When `isFirstMonitorAve` is set, the whole window is filled with the new sample. The flag is set:

- on a setpoint commit;
- on HV on and on HV off;
- at startup;
- in every HV-off cycle before the error-test delay expires.

It is cleared at the end of each cycle. Reproduce this reset; without it the display crawls toward a new setpoint for several seconds and looks like slow hardware.

## Wattage indicator banding

```

mwSafe      = 10
mwCaution  = (WattageMax - SafetyMargin * 2.0) * 1000
mwDanger    = WattageMax * 1000

```

| Range    | Colour | 4 W unit (mW) | 10 W unit (mW) |
|----------|--------|---------------|----------------|
| < mwSafe | white  | < 10          | < 10           |

| Range                | Colour     | 4 W unit (mW) | 10 W unit (mW) |
|----------------------|------------|---------------|----------------|
| mwSafe – mwCaution   | green      | 10 – 3900     | 10 – 9900      |
| mwCaution – mwDanger | yellow     | 3900 – 4000   | 9900 – 10000   |
| $\geq$ mwDanger      | red        | $\geq$ 4000   | $\geq$ 10000   |
| otherwise            | light blue | unreachable   | unreachable    |

- **mwSafe = 10 does not scale with the rating.** On a 10 W unit the white band covers only the bottom 0.1 % of the range.
- **A maximal setpoint shows yellow immediately.** mwCaution subtracts **twice** the safety margin, while the setpoint clamp (§9.1) subtracts it once, so the yellow band begins 50 mW below the highest commandable power.
- **The last branch is unreachable;** the source comments it “this should never happen, indicates error”.

**Range checking** HV\_InRange(set, mon, large) and I\_InRange(set, mon, large) both test:

```
err = 0.10 if large else 0.05
upper = set * (1 + err) + 1.0
lower = set * (1 - err) - 1.0
in range <=> lower < mon < upper          # strict
```

The monitor loop always uses large = true (10 %). The 5 % form is used only by the non-NSI setpoint correction (§9.2). The additive  $\pm 1.0$  is what makes low setpoints workable: at 5  $\mu$ A a pure 10 % band is  $\pm 0.5 \mu$ A, close to the offset seen on hardware (§10.3). **Do not tighten the band to a pure percentage.**

- **With HV on**, every cycle tests HVCheck against dblExpectedHV and ICheck against dblExpecteduA (§9.1). There is no debounce.
- **With HV off**, ErrTestCount counts cycles since HV was last switched on or off; both events reset it. For the first **ErrTestDelay = 7** cycles the tests are skipped and the average is kept primed. After that, both monitors are tested against an expected value of 1.1, giving the band  $-0.01 < \text{monitor} < 2.21 \text{ kV or } \mu\text{A}$ .

**The result only drives the “out of range” indicator.** The reference never switches HV off because of a range failure, a power excess or a missing MONX. It drops HV automatically only when:

- the interlock opens (§7.3);
- an I/O error occurs, through FsMessageBox  $\rightarrow$  FailSafe() (§9.2);
- the dialog closes (§9.2).

**Note.** An automatic HV trip on sustained out-of-range readings would be an addition to vendor behaviour, and this project does not add one. Anyone who does should debounce it in seconds rather than cycles and suspend it during setpoint changes. The reference avoids checking mid-ramp only because the monitor timer is stopped during a commit (§9.2).

## 7. GPIO, HV enable, and interlock

### 7.1 Reading pins

```
0x81 -> 1 byte, ADBUS
0x83 -> 1 byte, ACBUS
```

The reference reads the two ports in separate write/read round trips. Sending both opcodes in one write and reading 2 bytes also works (hardware). Output pins read back their driven state, so 0x81 can confirm that the enable bits actually changed. Mask ACBUS with 0x0F (§3).

Observed values, serial 01300036:

| Condition                         | ADBUS     | ACBUS     |
|-----------------------------------|-----------|-----------|
| HV off, idle, straight after init | 0001 1111 | 0000 0101 |
| HV on, 15 kV / 10 $\mu$ A         | 1111 1111 | 0000 0101 |

0x1F means CLK, DataOUT, DataIN, ADCS and DACS are high, so both chip selects are deasserted. Enabling sets 0x20 and 0x40, and MONX (0x80) follows. After other transactions, bits 0–1 vary (§3).

## 7.2 HV enable

**Source.** CTRL\_HV\_EN\_A (0x20) and CTRL\_HV\_EN\_B (0x40) are always set together and cleared together, each in a single 0x80 state write; the DACs hold their setpoints independently. The reference never reads the bits back. Its “HV enabled” indicator follows the software flag HV0n, not the pins. The on/off sequences are in §9.2.

**Recommendation.** Treat a state with only one bit set as a fault. Confirm every change by readback (§9.2).

**MONX (0x80, ADBUS7) is a live status input** (hardware). It reads 0 with HV off and 1 at 15 kV / 10  $\mu$ A, so it works as a “tube ready” confirmation. The reference only displays it and never acts on it. **Recommendation:** treat a MONX that has not asserted shortly after the setpoints are reached as a fault. The assert time, and whether MONX asserts at all with zero setpoints, have not been measured (§12).

## 7.3 Interlock

ACBUS0 (!RESET, 0x01): **1 = closed (safe to operate), 0 = open.** The source comments agree (“!RESET -> indicates interlock open”; status “0=open, 1=closed”) and the hardware reads 1 with the interlock closed. The open state has not been observed on hardware. An inverted read here fails dangerously, so check it on each unit before relying on a port.

**Source** (MonitorMiniXCtrlIO, 0nBnClickedHv0n):

- **On a transition to open:** disable all controls, dismiss any pending confirmation dialog by sending it an IDNO click, switch HV off (§9.2), then disable the controls again. The dialog step stops an operator from answering “yes” to an energize prompt raised before the interlock opened. **In a port, invalidate the in-flight confirmation; greying out the button is not enough.**
- **While open:** show “interlock open” and hold indInterlockClear at 3.
- **After a transition to closed:** HV is *not* re-enabled. “Interlock restored” shows for 3 monitor cycles. After that, the controls re-enable only if HV is off.
- **Switching HV on** is refused while the interlock is open. The interlock is checked again after the confirmation dialog returns.

The interlock is polled in the monitor loop, so the reference responds within about **1 s** (§6.4). During an NSI setpoint commit the commit loop calls the monitor routine directly, so polling continues. The non-NSI commit path does not poll for about 2 s. None of this is an asynchronous cutout; the hardware presumably enforces its own. Do not present the software interlock to users as instantaneous.

ACBUS2 (0x04) always reads set; its function is unknown. ACBUS3 is TSCS, **active high**, and is the only output in the high byte.

## 8. Temperature sensor (DS1722)

**Status.** The transactions below are from the source, and **both are confirmed on hardware** (§8.4). The first version of this document reconstructed a different read transaction, which failed. The conversion formula is from source and matches the DS1722’s 12-bit format.

## 8.1 Setup

Register map (source; the source calls 00h “Status”, but it is the configuration register’s read address):

| Read address | Write address | Register        |
|--------------|---------------|-----------------|
| 00h          | 80h           | Configuration   |
| 01h          | —             | Temperature LSB |
| 02h          | —             | Temperature MSB |

TSCMD = 0xE0 (“continuous convert, 8-bit res., no shutdown”). SetTempSensor() writes TSCMD + 0x08 = 0xE8: continuous conversion, 12-bit resolution, no shutdown. The write sequence is in §8.3.

The source configures the sensor only at startup (§5.2). The DS1722’s configuration is volatile and resets at power-up.

## 8.2 Conversion

Source (GetTemp):

```
raw = (msb << 4) + (lsb >> 4)           # 12-bit, left-justified in 2 bytes
if msb >= 0x80:
    raw -= 4096                           # sign extend
temp_c = raw * 0.0625
temp_f = temp_c * 9.0 / 5.0 + 32.0
```

0.0625 °C (1/16) is the standard DS1722 12-bit LSB. The reference caches the value as TemperatureC, shows it as "%.0f°C", and exposes it through its status API.

**Temperature is advisory in the reference.** Nothing acts on the value and no threshold exists. The reference reads it **every monitor cycle**, and a *communication* error on this read shuts everything down like any other I/O error (§9.2). A thermal cutout for unattended use would be your own engineering, and the tube’s specification should set its threshold.

**Recommendation.** A plausibility check alone cannot validate a reading. On hardware, a frame shifted by one bit decoded to a believable 12.5 °C (§8.4). Read three bytes starting at 00h and check the configuration byte as well: its top three bits always read 1, and after setup it should equal 0xE8.

## 8.3 Transactions

Source (ReadTemperature). TSCS rises with the clock **low**; the low state is written twice more; then the clock goes **high** for the transfer. The source comments: “take TS clock low - this makes clock high”.

```
0x86, 0x03, 0x00           # clock divisor
0x80, state & ~CLK & ~DATA, 0x7B # clock and data low
0x82, high | TSCS, 0x08     # assert TSCS
0x80, state & ~CLK & ~DATA, 0x7B # (again)
0x80, state & ~CLK & ~DATA, 0x7B # (again)
0x80, (state | CLK) & ~DATA, 0x7B # clock HIGH
0x12, 0x07, 0x01           # address 01h: 8 bits out
0x80, state, 0x7B           # "INPUTMODE" rewrite
0x20, 0x01, 0x00           # 2 bytes in: LSB, MSB
0x82, high & ~TSCS, 0x08   # deassert TSCS
```

Read 2 bytes, then GetTemp(rx[1], rx[0]). The source leaves the clock high afterwards.

Source (SetTempSensor). The data line is left as it was, and the 0x86 divisor is sent first:

```
0x80, state & ~CLK, 0x7B     # clock low
0x82, high | TSCS, 0x08     # assert TSCS
0x80, state & ~CLK, 0x7B     # (again)
```

```

0x80, state & ~CLK, 0x7B          # (again)
0x80, state | CLK, 0x7B           # clock HIGH
0x10, 0x01, 0x00, 0x80, 0xE8     # config write: 2 bytes out
0x82, high & ~TSCS, 0x08         # deassert TSCS

```

The source’s commented-out alternatives (other read opcodes; addresses 00h, 02h, 80h) suggest this path was tuned by experiment.

**Hardware.** With this clock sequence, address and data can be sent with 0x12 or 0x10, and the reply read with 0x20 or 0x24; all combinations give identical results (§8.4). 0x11 and 0x13 fail for both reads and writes. **This project** reads 3 bytes from 00h rather than 2 from 01h, so every reading carries the configuration byte for checking (§8.2). It also holds the data line low at select for the configuration write, as the hardware probe did.

## 8.4 Hardware status

Probe runs on 2026-09-17 compared the vendor clock sequence with holding the clock at one level throughout, across all output and read opcodes, at 1.5 MHz and 150 kHz (details in [hardware-notes.md](#)):

- **The vendor sequence works.** Reading from 00h, 01h and 02h returns e3 00 19, 00 19 and 19, as the register map requires. This holds with 0x10 or 0x12 for the address and 0x20 or 0x24 for the reply, at both clock rates.
- **The rate does not matter.** Every combination behaved the same at 150 kHz as at 1.5 MHz, so the failures below are not timing races. What matters is the clock level when TSCS rises and the level during the transfer.
- **Holding the clock at one level fails.** It is the first version’s reconstruction when held low with 0x10 and 0x20. One read edge returns the frame shifted by one bit, and no combination sends both 01h and 02h correctly.
- **Writes.** With the vendor sequence, writing 0xE4 and then 0xE8 reads back exactly, using 0x10 or 0x12; 0x11 and 0x13 do not write. The sensor then reads 0xE8 and gives live 12-bit readings (25.9 °C, then 27.3 °C). No write took effect with a one-level clock.

Before the first write, the config register read 0xE3: 9-bit resolution with shutdown set, assuming the bit layout 111, 1SHOT, R2–R0, SD. With shutdown set, the temperature register holds a stale value, which is why every reading should check the config byte (§8.2). The configuration is volatile, so it must be written again after every power-up.

## 9. Setpoint sequencing and safety logic

### 9.1 Clamping and the wattage limit

**Source** (OnBnClickedSethighvoltageandcurrentbutton). A commit runs these steps:

1. **Prepare.** Stop the monitor timer, disable the controls, show “Updating settings”, wait 20 ms, and prime the average (§6.4).
2. **Read** both fields with atof.
3. **Clamp** each value to [min, max], setting isOutOfRange on any adjustment.
4. **Detect changes** by comparing each value with PrevVoltage / PrevCurrent. These hold the integer part of the last value written, and HV off resets both to 0.
5. **Limit power.** If either value changed and volts \* microamps >= SafeWattageMW, reduce the **current**: microamps = SafeWattageMW / volts + 1e-9, and set isOutOfRange.
6. **Write back.** If isOutOfRange, rewrite **both** fields as "%.0f" and wait 200 ms so the operator sees what will be committed.
7. **Record expectations.** Set dblExpectedHV / dblExpecteduA from the clamped values, before any rounding.
8. **Write the DACs** in the order given in §9.2. Each per-channel handler **reads its value back from the field text** (§6.2).

**Step 5 always gives up current, never voltage**, in all three branches: both changed, only voltage changed, or only current changed. The voltage-reduction alternative is present but commented out, dated 20080507 with the note “only change current for wattage correction”. The operator’s chosen kV is kept and flux is given up. Keep this, and make the adjustment visible. Because the comparison is >=, exactly 3950 mW triggers a reduction.

**The written values can exceed the limit (source analysis).** Step 5 computes the current from the *unrounded* voltage, but step 6 rounds *both* fields and step 8 writes the rounded text.

- **Example, 4 W unit.** An entry of 20.51 kV with a request of 200  $\mu$ A is reduced to 192.59  $\mu$ A (3949 mW). The fields become “21” and “193”, and the DACs receive 21 kV  $\times$  193  $\mu$ A = **4053 mW, above the 4 W rating.**
- **Worst case on a 4 W unit** is about 4.05 W. It needs a fractional voltage entry plus a clamp or a reduction.
- **On a 10 W unit** (derived, with WattageMax = 10) a reduction only happens at 49.75 kV or above, so the worst case is exactly 10.00 W.
- **Detection.** WattInRange (below 4050 mW) does not flag it; the banding shows red.

This has not been exercised on hardware.

### Recommendations beyond the reference:

1. **Apply the power check to the values that will actually be written**, after any rounding, and round current down rather than to nearest.
2. **Bound-check the rating at startup**, not only the setpoint at commit time (§6.1.1).
3. **Recompute SafeWattageMW from WattageMax at every commit** rather than caching it, so a configuration reload can never leave a stale limit.

The clamp works on the product of the clamped setpoints, so on a 10 W unit the 50 kV  $\times$  200  $\mu$ A corner is reachable within 50 mW. **The 10 W unit is the first configuration in which DAC full scale and the power limit coincide.** Test the top of the range deliberately before operating there.

## 9.2 Sequences

**Setpoint commit, NSI units (source)** This path applies to serial > 9999, including this project’s unit. Here SetI and SetV are the per-channel handlers of §6.2, and “wait for X” means repeating 500 ms + one monitor cycle + 20 ms, up to 15 times, until X is in range with large = true:

```
if new kV > previous kV and new kV * previous  $\mu$ A >= SafeWattageMW:
    250 ms; SetI; wait for current
SetV; wait for HV
250 ms; SetI; wait for current
500 ms
```

The current goes first **only when** raising the voltage with the old current would reach the power limit. Otherwise the voltage goes first. The waits give up silently after 15 tries, about 8 s each, with no error.

### Setpoint commit, non-NSI units (source)

```
if new kV > previous kV: SetI; 1 s; SetV; 1 s
else: SetV; 1 s; SetI; 1 s
one monitor cycle; 20 ms
for each channel whose monitor is within the 5 % band:
    write (2  $\times$  setpoint - monitor) # one-step correction
```

The non-NSI path has not been exercised on hardware.

In both paths the commit ends by restarting the monitor timer and re-enabling the controls. **Recommendation:** order the two DAC writes so that no intermediate state exceeds the power limit, and wait for each channel to settle. Both vendor paths do this, in different ways.

### Energizing (source, OnBnClickedHv0n)

```
ErrTestCount = 0; prime average
refuse if interlock open
confirmation dialog (skipped when running as the DLL)
refuse if interlock open, or the answer is no
```

```
write both enable bits; HVOn = true
setpoint commit (§9.1)
```

**The reference sets the enables first and then commits the setpoints.** The DACs are at zero at that moment, because both startup and HV off leave them there, and PrevVoltage is 0. The commit therefore ramps up from zero under the ordering rules above. The reference neither reads back the enable bits nor checks MONX.

**Recommendation.** Keep the vendor order, but make its precondition explicit: before enabling, **write zero to both DACs** instead of assuming they are zero. After enabling, confirm the bits by readback, commit the setpoints, and then require MONX to assert (§7.2). MONX is checked after the ramp because it is not known whether it asserts with zero setpoints. Enabling with the DACs already at the target would apply full HV in one step instead of ramping.

#### De-energizing (source, OnBnClickedHvOff)

```
prime average; HVOn = false; disable controls; stop monitor timer
HV DAC = 0; NSI only: 200 ms
current DAC = 0; NSI only: 100 ms
ErrTestCount = 0; restart monitor timer
clear both enable bits
NSI only: 500 ms; re-enable controls
```

The DACs go to zero **before** the enables are cleared. The reference does not read the bits back. **Recommendation:** read ADBUS back and confirm 0x20 and 0x40 are clear. If they are still set, tell the operator to check the tube physically rather than retrying silently.

**Failsafe and shutdown (source)** Any I/O error (DAC write, ADC read, GPIO read, temperature read) calls FsMessageBox. That routine runs FailSafe() once, logs the message, and closes the application. Closing the dialog runs FailSafe() too. FailSafe() then does the following:

1. Switch HV off as above.
2. Stop the monitor.
3. Set both fields to “0” and run the per-channel handlers. These clamp to the minima, so the DACs are left at **10 kV and 5  $\mu$ A**, with the enables off.
4. If neither DAC write failed, disable MPSSE (§5) and close the device.

**Recommendation.** For an emergency stop, clearing the enables first is the fastest way to remove HV. This project’s device layer does that and then zeroes both DACs; normal switch-off follows the DACs-first order above.

## 10. Empirical validation

### 10.1 Clock edge determination

An exhaustive sweep of the two plausible nibble-out opcodes against the two plausible byte-in opcodes, at a known setpoint of 15.0 kV / 10.0  $\mu$ A (1200 and 200 counts commanded):

| Nibble | Read | ch0 / HV    | ch1 / I    | Verdict       |
|--------|------|-------------|------------|---------------|
| 0x13   | 0x20 | <b>1204</b> | <b>208</b> | correct       |
| 0x13   | 0x24 | 1187        | 192        | ~1–1.5 % low  |
| 0x12   | 0x20 | 184         | 209        | ch0 collapsed |
| 0x12   | 0x24 | 216         | 188        | ch0 collapsed |

**0x13 (bits out) with 0x20 (bytes in) is correct**, as in the source. The null bit was clear in all four cases, and the 13-bit unpack is validated (0x25A2 >> 3 = 1204, 0x0680 >> 3 = 208). Both correct replies also satisfy the trailing-bit rule (§6.3).

The two failure modes look different:

- **Wrong read edge (0x24)** samples slightly stale data and loses the low bits: a small negative bias that is easy to mistake for a calibration error.
- **Wrong nibble edge (0x12)** corrupts ODD/SIGN, so the channel select fails and ch0 returns roughly ch1's value. Both channels still look plausible, so a nonzero test cannot catch it. **Only a cross-check against a known setpoint can.**

## 10.2 Monitors do not zero with HV off

With HV off, the monitors read 8–23 counts in August and 0–20 counts, including exact zeros, in September. That is ADC offset and dither, not signal. Consequently:

- **A test keyed on exact zero proves nothing.** It will report a dead read path as working, and a zero reading is normal. The August sweep script's Stage A verdict made this mistake; its Stage B is what actually discriminated.
- **A GUI must not treat a few counts on an unenergized tube as a fault**, and must not treat nonzero monitors as proof that HV is on. **Use MONX and the enable-bit readback to tell whether HV is on.**

## 10.3 Readback accuracy

At 15 kV / 10  $\mu$ A the HV channel read 1204 counts against 1200 commanded (+0.33 %), and the current channel read 208 against 200 (+4 %). At 10  $\mu$ A the current channel uses only 200 of 4000 counts, so 8 counts is offset and quantization rather than a real error. The additive  $\pm 1.0$  in the range band (§6.4) absorbs it.

## 10.4 Other confirmations (August 2026)

- The MPSSE sync check works: 0xAB  $\rightarrow$  0xFA 0xAB.
- Both DAC channels are write-and-update. 0x10 was used throughout; 0x11 was not tested.
- The interlock read CLOSED throughout; the open path was **not** exercised (§12).
- The enable-bit readback at teardown returned clear on every run.

## 10.5 Rating provenance

The 10 W rating of serial 01300036 comes from the unit's hardware documentation, **not** from the device or the reference software, neither of which reports it. The only energized measurement so far is 15 kV / 10  $\mu$ A: 150 mW, 1.5 % of the rating. Nothing in this document validates behaviour near either variant's power limit, and the clamp arithmetic (§9.1) and banding (§6.4) are unexercised above 150 mW.

## 10.6 Read-only checks (September 2026)

These runs used this project's code, with HV off throughout:

- The device identifies as bcdDevice 0x0500, and the sync check works.
- Idle GPIO values match §7.1; the enable bits read back clear; MONX is 0; the chip selects are idle between transactions.
- ACBUS bits 4–7 read at random (§3).
- The ADC trailing bits follow the B1–B3 rule in all 412 readings (§6.3).
- The DS1722 results are in §8.4.

The full log is in [hardware-notes.md](#).

---

## 11. Transaction serialization

Every peripheral shares one clock and one data line, the host manages the chip selects, and there is no hardware arbitration. **Two overlapping transactions will corrupt each other.** Because the DAC path is one of them, the result can be a wrong setpoint, not merely a bad reading.

**Source.** The reference guards each write with a spin lock:

```
while(port_busy);
port_busy = 1;
status = Write(tx, pos, &ret_bytes);
port_busy = 0;
```

The test-and-set is not atomic. In the ADC and DS1722 paths, and in the GPIO path, which takes the lock separately for the write and for the read, nothing holds the lock across the whole write-then-read pair. This works only because the reference is single-threaded, driven by a message-pump timer.

A Python GUI with a monitor loop, operator-initiated commits and a temperature poll has several independent sources of traffic. Requirements:

1. **Serialize whole transactions**, write and read together. The simplest way is to route all device access through one worker thread with a command queue, which is what this project does. Otherwise, hold one lock across each write-then-read pair.
2. **Purge before any transaction whose reply you will parse.** The reference never purges; stale bytes in the RX buffer would misalign every later read.
3. **Keep device I/O off the UI thread.** A 40 ms timeout plus a 4 ms latency timer per transaction, several times per cycle, is enough to make a single-threaded GUI feel unresponsive.
4. **Never let the shutdown path be blocked.** A stuck monitor must not prevent HV from being dropped. With a worker thread, give the emergency stop the highest priority and bound every I/O call with a timeout. With a lock, bound every acquisition with a timeout and treat expiry as a reason to force the enables clear. Never write to the device from a second thread while another write may be in progress: interleaved streams can turn one command's bytes into DAC data.

---

## 12. Known gaps

These items are not settled; they are listed so that nobody mistakes them for facts.

| Item                             | Status                                                                                                                                                                            |
|----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| DS1722 write after power-up      | Confirmed with the probe. This project's own <code>configure_temperature_sensor()</code> sends the same bytes but has not yet been run on a freshly powered, unconfigured sensor. |
| DS1722 config 0xE3               | The meaning assumes the datasheet bit layout; unverified. How the register reached that value is unknown.                                                                         |
| ADC part number                  | Behaves like an MCP3202/LTC1298-type part, not a MAX186 (§6.3). Unconfirmed; irrelevant to the implementation.                                                                    |
| Clock inversion on the board     | Inferred from source comments and consistent with the DS1722 results (§8.4); not measured.                                                                                        |
| ACBUS2 (0x04)                    | Always reads set. Function unknown.                                                                                                                                               |
| Interlock-open behaviour         | Never exercised on hardware. The state machine is taken from source, not validated. <b>Test it deliberately before relying on it.</b>                                             |
| MONX assert time                 | Not measured, nor whether MONX asserts with zero setpoints. The 1 s timeout used by this project is a guess.                                                                      |
| Double startup (§5.2)            | Present in source; reason unknown.                                                                                                                                                |
| Non-NSI (Comet) path             | Setpoint correction and timing (§9.2) are from source only; no non-NSI unit tested.                                                                                               |
| Power overshoot through rounding | Found by source analysis (§9.1); not exercised.                                                                                                                                   |
| 0x11 DAC opcode                  | Untested; 0x10 confirmed.                                                                                                                                                         |

| Item                    | Status                                                                                                                                                                                |
|-------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 40 kV board             | All validation used a 50 kV unit (sn 01300036). The 40 kV constants are from source only.                                                                                             |
| 4 W variant             | Never exercised. The 4 W constants are from source.                                                                                                                                   |
| 10 W constants          | <b>Derived, not transcribed</b> (§6.1.1). The 10 W rating of sn 01300036 is confirmed from its documentation, but the derived limits have not been exercised at the top of the range. |
| Safety-margin scaling   | SafetyMargin is absolute (50 mW), giving the two variants different proportional headroom (§6.1.1). Whether the vendor intended it to scale is unknown.                               |
| FT2232 EEPROM user area | Never read. May or may not contain a rating (§2.1).                                                                                                                                   |
| _RELEASE_1uA build      | Lowers CurrentMin to 0.999999 (50 kV table only). Not exercised.                                                                                                                      |
| Thermal limits          | No threshold exists in the reference. Any cutout is your own engineering judgement.                                                                                                   |

Settled in this revision: NumAVE = 7, ErrTestDelay = 7 (with its actual meaning, §6.4), indInterlockClear = 3, the range tolerances (10 % / 5 %), the monitor period (1000 ms), the NSI and non-NSI setpoint timing (§9.2), and the presence of the TSCS leading assert (§8.3).

## 13. Revision history

**13 August 2026.** First version, from an excerpt of MiniXDlg.cpp and the August hardware sweep.

**17 September 2026.** Checked against the full DLL source and September hardware runs.

Corrections:

- **§6.4 range checking.** The range check only drives an indicator; the reference never switches HV off because of it. ErrTestDelay delays the *HV-off* tests after HV is switched on or off; it is not a debounce on sustained out-of-range readings. The tests use the unrounded monitor values, not the rounded integers.
- **§9.2 energizing.** The reference sets the enables first and then commits the setpoints, ramping from zero. The first version said the opposite and presented it as the reference’s rule.
- **§9.2 ordering.** On NSI units the current goes first only when raising the voltage with the old current would reach the power limit, and each write waits for its monitor to settle (up to  $15 \times 500$  ms). The 200 ms “settling delay” in the first version is actually the pause after the fields are rewritten (§9.1). Non-NSI units use 1 s delays and a one-step correction.
- **§5 initialization.** The reference does not purge, and the clock divisor is not part of MPSSE init.
- **§5.2 startup.** The reference runs the startup block twice, 2 s apart.
- **§8 DS1722.** The read and setup transactions are now taken from source, and both are confirmed on hardware. The first version’s reconstruction failed on hardware.
- **§1, §6.3 ADC part.** The “MAX186-family” label does not match the part’s behaviour.
- **§11 serialization.** The first version said the reference purges “at several points”; it never does.

Additions:

- **§2.1 Comet (non-NSI) handling** and the default 40 kV table.
- **§3 ACBUS** bits 4–7 are not pins and must be masked; ADBUS bits 0–1 readback varies with the previous transaction.
- **§6.2** How setpoints reach the DAC, the DAC routine’s refusal of out-of-range values while HV is on, and float truncation.
- **§6.3** The trailing ADC bits repeat B1–B3, with the extra framing check this allows.

- §6.4 Monitor period, per-cycle order, the averaging reset conditions, and `WattInRange`.
- §7.2 The reference's indicator follows its software flag.
- §7.3 Polling during commits.
- §9.1 Power overshoot through rounding, up to about 4.05 W on a 4 W unit.
- §9.2 Failsafe on any I/O error, and DACs left at their minima on close.
- §10.6 The September read-only checks.